Practice 09: Inventory Dashboard — a beginner's React workflow

This guide explains how to plan and build the mini project in Practice Lab 09 requirements. Read the requirements first; they define the assignment. Component names and implementation choices below are suggestions, not extra requirements.

You are building a dashboard for a small shop. A user should see its products, add a product, change stock availability, and remove a product. You will start a **new Vite React + TypeScript project**. This repository's TaskFlow app is a learning reference; the inventory project is not an extension of it.

The examples below explain individual pieces. They are not a complete application to paste into one file.

1. What must we build?

Finish and test CORE before adding extras.

CORE feature	What the user should experience
Fetch products	Opening the app loads the supplied JSON file.
Loading state	A message appears while the app waits for data.
Error and Retry	A failed load shows a clear message and a working Retry button.
Product display	Every product shows its name, category, price, and stock status.
Summary	At least one calculated number, such as total products, updates with the list.
Add	A form accepts at least a name, category, and price and adds the product immediately.
Toggle stock	A button changes a product between in stock and out of stock.
Remove	A button removes a product from the list.
Usable styling	The layout is readable, including at a narrower browser width.
Build	`npm run build` completes without errors.

Search, stock filters, a no-matches message, additional summaries, and editing a product's name or price are **optional**.

There is no database in this assignment. Fetching reads the initial file. Adding, toggling, editing, and deleting change React state in memory; they do not rewrite the JSON file. Reloading the page loads the original products again.

2. Plan together before writing code

Use the first 15 minutes to answer these questions:

1. What can change? The product collection, the loading/error state, and the values being typed into the form.
2. What sections do we need? A header, summary area, add form, loading/error messages, and product list.
3. What can we calculate? The total number of products comes from the list; it does not need separate state.
4. Who owns each file? Agree on component props and callback names before splitting the work.

A possible split is: one person handles `App` and loading, one builds the form, one builds product cards, and one handles styling and acceptance checks. Adapt this to your team size. Integrate early so each component works with the others.

Suggested timing from the assignment:

- **0–15 minutes:** read and plan.
- **15–120 minutes:** implement CORE.
- **120–175 minutes:** test, fix bugs, improve styling, then attempt extras if time remains.
- **Last 5–10 minutes:** complete the checklist and team reflection.

3. Create the new project and place the data

Use the Vite setup process taught in Session 05. Run the following from the folder where you keep your projects, outside the existing TaskFlow project:

```
npm create vite@latest inventory-dashboard -- --template react-ts
cd inventory-dashboard
npm install
npm run dev
```

`npm` runs project tools and installs dependencies. `react-ts` selects React with TypeScript. Open the local URL printed by the dev server. Leave that terminal running while working; use another terminal for build commands. Press `Ctrl+C` to stop the server.

Check `node --version` and `npm --version` if setup fails. The latest Vite requires a compatible Node.js version; follow its setup warning and the official Vite setup instructions, or use the version specified by your instructor.

Remove the generated demo UI and write your own components and stylesheet. Do not copy the TaskFlow application into the new project.

The PDF names a supplied file, `assets/products.json`. **That file is not present in this repository at the time this guide was written.** Obtain the supplied file from the instructor or class materials; the single example object in the PDF is not the full dataset.

Copy the supplied file to this location in your new project:

```
inventory-dashboard/
├── public/
```

```
│   └── products.json
├── src/
│   ├── components/
│   │   ├── Header.tsx
│   │   ├── StatCard.tsx
│   │   ├── AddProduct.tsx
│   │   └── ProductItem.tsx
│   ├── App.tsx
│   ├── main.tsx
│   └── index.css
├── index.html
└── package.json
```

Vite serves files inside `public` from the URL root. Therefore, use `fetch("/products.json")`, not `fetch("/public/products.json")`. Open `/products.json` on your dev server to check that it returns the product data. See Vite's public directory documentation.

4. Understand how React starts

```
index.html → src/main.tsx → App → child components
                                ↘ index.css styles the page
```

`index.html` contains a `<div id="root">`. `main.tsx` connects React to that element and renders `<App />`; it also imports the stylesheet. `App.tsx` describes the dashboard and joins its components together.

A **component** is a function that returns a piece of the interface. Its name starts with a capital letter, such as `Header`. A `.tsx` file can contain TypeScript and JSX. **JSX** is HTML-like markup inside JavaScript, such as `<h1>Inventory Dashboard</h1>`. Curly braces insert JavaScript values: `<p>{totalCount}</p>`. Use `className` to connect JSX elements to CSS classes.

Rendering means React calls components to work out what the interface should look like. When state changes, React renders again and updates the relevant parts of the page. You do not manually edit the page with DOM queries.

5. Give each component one clear responsibility

```
App — owns the products and coordinates changes
├── Header — displays the dashboard title
├── StatCard — displays a calculated summary
├── AddProduct — collects and validates a new product
└── ProductItem — displays one product and its action buttons
    (App renders one ProductItem for each product)
```

Component	Data received through props	Responsibility and local state
`App`	None required	Owns products, loading/error state, fetch/retry, and collection update functions.
`Header`	None required	Shows the page title; no state needed.
`StatCard`	`label: string`, `value: number`	Displays a number calculated by `App`; no state needed.
`AddProduct`	`onAddProduct(name, category, price)` callback	Owns draft inputs and form errors; asks `App` to add a valid product.
`ProductItem`	`id`, `name`, `category`, `price`, `inStock`, `onToggle`, `onDelete`	Shows one product; sends its ID back when an action is clicked. Optional editing would add local draft state.

A separate `ProductList` component is optional. Keeping the list's `.map()` inside `App` is enough for CORE. An inventory product already contains its category; this project does not need TaskFlow's users request or `PersonSummary` component.

6. Props, state, and derived values

Props are inputs a parent passes to a child. A child reads them through `props.name`; it should not change them directly.

State is a component's memory for information that changes. `useState` supplies a current value and a setter:

```
const [isLoading, setIsLoading] = useState(true);
```

`isLoading` starts as `true`. Calling `setIsLoading(false)` asks React to render with the new value. Calling a setter does not immediately replace the variable in the already-running event handler.

Describe a product in `App.tsx` using a TypeScript interface:

```
interface Product {
  id: number;
  name: string;
  category: string;
  price: number;
  inStock: boolean;
}
```

An **interface** describes the fields and their types. `boolean` means `true` or `false`; `Product[]` means an array of products. Types help catch mistakes during development; they do not check downloaded JSON at runtime.

Inside `App`, start with:

```
const [products, setProducts] = useState<Product[]>([]);
const [isLoading, setIsLoading] = useState(true);
const [hasError, setHasError] = useState(false);

const totalCount = products.length;
```

Import `useState` and `useEffect` from `"react"` when using them.

Information	Where it belongs	Why
Products	`App` state	Loading and user actions change the collection.
Loading/error flags	`App` state	The request changes what the page should show.
Name/category/price being typed	`AddProduct` state	Only the form needs these unfinished values.
Form error	`AddProduct` state	Validation changes the feedback.
Total products	Calculate in `App`	It is always `products.length`.
Search and selected stock filter, if attempted	`App` state	They control which products appear.
Visible products, if filtering is attempted	Calculate in `App`	They follow from products and the selected controls.

Keeping the collection in the parent is called **lifting state up**: both the form and product cards need to affect it, so their common parent owns it. Do not store another copy of the product list in each child.

7. Load products and handle failure

The page-load workflow is:

```
App first renders with an empty array and a loading message
    ↓
useEffect starts the load function
    ↓
fetch requests /products.json → response.json() reads the data
    ↓
Success: save products        Failure: set the error flag
    ↓                            ↓
Stop loading                 Stop loading and show Retry
```

```
    ↓                              ↓
Show product cards           Retry calls the load function again
```

This example belongs inside `App`, after the state declarations:

```tsx
async function loadProducts() {
  setIsLoading(true);
  setHasError(false);

  try {
    const response = await fetch("/products.json");
    const data = (await response.json()) as Product[];
    setProducts(data);
    setIsLoading(false);
  } catch {
    setHasError(true);
    setIsLoading(false);
  }
}

useEffect(() => {
  loadProducts();
}, []);
```

`async` allows `await` inside a function. `await` waits for a result without freezing the interface. `try/catch` provides a failure path when the request or JSON parsing throws an error. `as Product[]` tells TypeScript the expected shape; it does not transform or validate the file.

`useEffect(..., [])` starts the request after the component mounts. An empty dependency array means it does not rerun just because you type or update a product. A generated starter using React Strict Mode can run an extra setup cycle during development, so do not interpret this as a guarantee of exactly one network request.

Keep the effect callback itself synchronous, as shown; it calls the async function. Do not fetch directly in the component body, where every render would start another request.

Use **conditional rendering** to choose the UI:

```tsx
{isLoading && <p>Loading products...</p>}

{!isLoading && hasError && (
  <div>
    <p>We could not load the products. Please try again.</p>
    <button onClick={loadProducts}>Retry</button>
  </div>
)}
```

Here `&&` means "show this when the condition is true." Only show the loaded list when `!isLoading && !hasError`. Show the add form after a successful load as well, so a pending request cannot replace products the user has just added.

A missing file does not necessarily make `fetch()` throw: HTTP error responses still produce a response object. In the assignment's rename test, the missing-file response typically contains non-JSON content, so `response.json()` throws and reaches `catch`. This small example follows that class pattern; it is not a general-purpose HTTP or data validator.

8. Display products and pass typed props

Define a props interface in `ProductItem.tsx`:

```
interface ProductItemProps {
  id: number;
  name: string;
  category: string;
  price: number;
  inStock: boolean;
  onToggle: (id: number) => void;
  onDelete: (id: number) => void;
}
```

`(id: number) => void` describes a function that accepts an ID and returns no useful value. Inside `function ProductItem(props: ProductItemProps)`, read values with `props.name`, `props.price`, and so on. Display status text with:

```
<span>{props.inStock ? "In Stock" : "Out of Stock"}</span>
```

The `? :` expression chooses between two values. Include readable text as well as different stock colors. Agree on a currency label rather than assuming the dataset specifies one.

In the successful-load part of `App`'s JSX, render the collection:

```
<ul className="product-list">
  {products.map((product) => (
    <ProductItem
      key={product.id}
      id={product.id}
      name={product.name}
      category={product.category}
      price={product.price}
      inStock={product.inStock}
      onToggle={handleToggleProduct}
      onDelete={handleDeleteProduct}
    />
```

```
    ))}
  </ul>
```

This snippet assumes you have imported `ProductItem` and defined the handlers described below. Have `ProductItem` return an `<li>` for this list structure.

`.map()` turns each product into one component. `key` gives React a stable identity for each item. It must be unique; React uses it internally, so pass `id` separately when the child needs it. Avoid array positions as keys because removing products changes their positions.

A summary uses the same parent data:

```
<StatCard label="Total Products" value={totalCount} />
```

9. Add a product: follow the full round trip

A **controlled input** gets its displayed value from state and updates that state through `onChange`.

Inside `AddProduct`, keep drafts as strings, including the price while the user is typing:

```
const [draftName, setDraftName] = useState("");
const [draftCategory, setDraftCategory] = useState("");
const [draftPrice, setDraftPrice] = useState("");
const [formError, setFormError] = useState("");
```

For example, a name field can be:

```
<label htmlFor="product-name">Product name</label>
<input
  id="product-name"
  value={draftName}
  onChange={(event) => setDraftName(event.target.value)}
  required
/>
```

Use corresponding labeled fields for category and price. For price, `type="number"`, `min="0"`, `step="0.01"`, and `required` provide helpful browser validation. Input values still arrive as strings, so convert the price with `Number(draftPrice)` on submission. Check for an empty price before conversion because `Number("")` is `0`.

The form's props can be:

```
interface AddProductProps {
  onAddProduct: (name: string, category: string, price: number) => void;
```

```
}
```

Use `function AddProduct(props: AddProductProps)`. Put submission logic in a handler connected to `<form onSubmit={handleSubmit}>`, with an Add button using `type="submit"`. If typing the handler explicitly, import `FormEvent` using `import type { FormEvent } from "react"`.

The submission workflow is:

1. Call `event.preventDefault()` so submitting does not reload the page.
2. Trim the name and category to remove outside spaces.
3. Reject blank names/categories and a blank or invalid price. Show a helpful form error and return early. Requiring a nonnegative price is a suggested validation rule, not an additional PDF requirement.
4. Call `props.onAddProduct(name, category, price)` with the cleaned values.
5. `App` creates a product with a unique ID. Choosing `inStock: true` initially is a reasonable team decision; the PDF does not specify a default.
6. `App` calls `setProducts` with a new array containing the new product.
7. React renders the updated list and recalculates the summary.
8. Clear the form drafts and error after a successful submission.

The parent connects the form with `<AddProduct onAddProduct={handleAddProduct} />`.

This is a **callback prop**: the parent gives a function to the child, and the child calls it to request a change. Props carry information down; callbacks let a child's action reach the parent.

10. Update the collection without changing the old array

An **immutable update** creates a new array, and a new object for any changed product. This keeps the previous state intact and gives React the next state to render.

The following handlers belong inside `App`:

```
function handleAddProduct(name: string, category: string, price: number) {
  setProducts((currentProducts) => {
    const highestId = currentProducts.reduce((highest, product) => {
      return product.id > highest ? product.id : highest;
    }, 0);

    const newProduct: Product = {
      id: highestId + 1,
      name: name,
      category: category,
      price: price,
      inStock: true,
    };

    return [...currentProducts, newProduct];
  });
}
```

```
function handleToggleProduct(id: number) {
  setProducts((currentProducts) =>
    currentProducts.map((product) => {
      if (product.id === id) {
        return { ...product, inStock: !product.inStock };
      }

      return product;
    }),
  );
}

function handleDeleteProduct(id: number) {
  setProducts((currentProducts) =>
    currentProducts.filter((product) => product.id !== id),
  );
}
```

The setter's function receives the latest collection. This is still `useState`, not a separate React feature or state library.

- **Add:** `[...currentProducts, newProduct]` copies existing items into a new array and appends the new item.
- **Toggle:** `.map()` keeps every item but replaces the matching object. `{ ...product }` copies its fields; `!product.inStock` reverses its boolean value.
- **Delete:** `.filter()` keeps only items whose IDs do not match the selected ID.
- **ID calculation:** `.reduce()` carries the highest ID through the array. Adding one avoids collisions with existing products, assuming the supplied IDs are unique. `products.length + 1` can collide after deletion and should not be copied from the current TaskFlow implementation.

Inside `ProductItem`, buttons call the callback with this product's ID:

```
<button onClick={() => props.onToggle(props.id)}>
  {props.inStock ? "Mark Out of Stock" : "Mark In Stock"}
</button>
<button onClick={() => props.onDelete(props.id)}>Remove</button>
```

The arrow function waits for a click. Writing `onClick={props.onDelete(props.id)}` would call the function while rendering instead.

For a toggle, the complete path is: **click → child callback → parent handler → new collection → render → updated stock label**. The total number of products stays the same; an optional in-stock summary would change. For removal, the card disappears and the total decreases.

11. Add optional features only after CORE works

Search and stock filters

Keep `searchText` and `stockFilter` in `App` state. Define the filter choices with a union type:

```
type StockFilter = "all" | "in-stock" | "out-of-stock";

const [searchText, setSearchText] = useState("");
const [stockFilter, setStockFilter] = useState<StockFilter>("all");

const visibleProducts = products.filter((product) => {
  const matchesSearch = product.name
    .toLowerCase()
    .includes(searchText.toLowerCase());

  const matchesStock =
    stockFilter === "all" ||
    (stockFilter === "in-stock" && product.inStock) ||
    (stockFilter === "out-of-stock" && !product.inStock);

  return matchesSearch && matchesStock;
});
```

Connect search and filter controls to their setters, and map `visibleProducts` instead of `products`. Both conditions must pass. Keep the original collection in state; do not overwrite it with the filtered list or hidden items will be lost.

When loading has succeeded and the visible list is empty, show “No products match your search or filter.” A newly added product might be hidden by an active filter, so clear the controls when testing whether adding works.

Additional summaries

Calculate these from `products`, without adding state:

```
const inStockCount = products.filter((product) => product.inStock).length;
const outOfStockCount = products.length - inStockCount;
```

If you sum prices with `.reduce()`, label the result precisely. This dataset has no quantity field, so it cannot tell you the value of all physical units in the shop. You can show the sum of listed prices or the sum of in-stock listed prices and explain that meaning.

Keep dashboard totals based on all products unless you explicitly label them as filtered totals.

Edit name or price

Let `ProductItem` own `isEditing`, draft inputs, and an edit error. On Edit, initialize the drafts from the current props. On Save, validate and call a parent callback such as `onSaveEdit(id, name, price)`; the parent uses `.map()` and object spread to replace the matching fields. Cancel closes the editor without changing the collection, and reopening should start with the saved values.

12. Style a usable dashboard

Write your own `src/index.css` and import it from `main.tsx`. Build a simple, readable design before adding decoration:

- Put the dashboard in a centered container with padding and a maximum width.
- Give the header and summary area clear visual emphasis.
- Use visible labels for every form field; placeholders alone disappear during typing.
- Make product names, categories, prices, and stock labels easy to scan.
- Use different stock colors together with status text, so color is not the only explanation.
- Give buttons and inputs consistent spacing, borders, and visible keyboard focus.
- Let rows wrap or stack at narrow widths; avoid fixed widths that force horizontal scrolling.

A media query can change the layout for smaller screens. Check the form, long product names, action buttons, and error messages after narrowing the browser.

13. Build and test in small steps

A practical implementation order is:

1. Start Vite and replace the demo with your header and page container.
2. Agree on product fields, state ownership, and props.
3. Fetch the supplied data and implement loading, error, and Retry.
4. Display all product fields using `.map()` and unique keys.
5. Calculate and display total products.
6. Implement the controlled add form and parent callback.
7. Implement stock toggle and removal.
8. Finish readable, responsive CSS.
9. Run the CORE checks below and fix failures.
10. Attempt optional features, then repeat the affected checks.

CORE acceptance checklist

- [ ] Open the app and confirm products come from the supplied JSON, not a hardcoded array.
- [ ] Confirm a loading message appears while the request is pending. Local files can load quickly; browser Network throttling can help make this visible.
- [ ] Temporarily rename `public/products.json`, reload, and confirm an error message and Retry appear.
- [ ] Restore the exact filename and click Retry without reloading; confirm products appear and the error clears.
- [ ] Check that every product displays its name, category, price, and stock status.
- [ ] Compare the summary to the number of products displayed with no optional filters active.
- [ ] Add a valid product; confirm it appears immediately and the total increases by one.
- [ ] Try blank fields and an invalid price; confirm the form provides useful feedback.
- [ ] Toggle a product twice; confirm the stock text changes both ways and its other fields stay the same.
- [ ] Remove a product; confirm it disappears and the total decreases by one.
- [ ] Remove a middle product, add another, then toggle/remove the new one; confirm only the intended product changes. This helps catch duplicate IDs.
- [ ] Remove all products; confirm the total becomes zero and adding still works.

- [ ] Narrow the browser; confirm the form and product actions remain usable.
- [ ] Run `npm run build` from the new project's folder and resolve all errors.

Build checks catch compilation problems, but they do not prove that user actions work. Perform the browser checks as well. If your generated project provides `npm run lint`, run it to catch additional code issues. Use `npm run preview` after a successful build to inspect the built app locally.

Common problems and where to look

Symptom	What to check
Loading never finishes	Set loading to false in both success and failure paths.
Fetch returns HTML or JSON parsing fails	Check the filename, `public` location, and `/products.json` URL.
Form submission reloads the page	Call `event.preventDefault()` in the submit handler.
Typing does not update the field	Connect both `value` and `onChange` to state.
Two cards change together	Check product IDs for duplicates.
A click handler runs immediately	Pass a function to `onClick`; do not call it during render.
Summary is stale	Derive it from the current collection instead of storing a second count.
Blank page	Read the terminal and browser console for import, export, JSX, or runtime errors.
Changes disappear after refresh	Expected: this exercise keeps changes only in React state.

14. Stay within the concepts taught through Session 09

Use components, typed props, `useState`, controlled inputs, event handlers, conditional rendering, `.map()`, `.filter()`, `.reduce()`, derived data, callback props, immutable collection updates, `useEffect` with `[]`, `async/await`, and `try/catch` with loading/error/Retry.

The assignment explicitly excludes:

- Effects with any dependency array other than `[]`.
- Props destructuring: use `props.name`, not `function ProductItem({ name })`.
- React Fragments: use an appropriate wrapper such as `<div>` or `<li>`, not `<>...</>`.
- A separate shared types file: keep interfaces in the component files where they are needed.
- Array methods or browser APIs not taught in class.
- `localStorage`, routing, Context, custom hooks, and state-management libraries.

Do not add these to solve an optional extra. A small, complete CORE project meets the assignment.

15. Connect this to the class project

These existing files explain familiar patterns. Read them to understand the ideas, then write the inventory components yourselves:

Class reference	What to learn for the new project
`src/main.tsx` and `index.html`	How React connects to the page.
`src/App.tsx`	Parent state, fetch/retry, derived values, and collection handlers.
`AddTask.tsx`	Controlled drafts, validation, and an add callback.
`TaskItem.tsx`	Child action buttons and local edit state.
`StatCard.tsx`	A reusable display component with typed props.
`Header.tsx` and `SectionTitle.tsx`	Small components that organize the interface.
`PersonSummary.tsx`	Displaying calculated data through props; a people section is unnecessary here.
`src/index.css`	Examples of spacing, classes, status styles, and a media query.

16. Finish with a team reflection

Write your own short answers after testing:

1. What information did you store as state, and why?
2. What did you calculate from other data instead?
3. Which components did you create, and what does each do?
4. Where did a child action need to change parent state? Trace one example.
5. What was the hardest bug, and how did you find it?
6. What would you improve with 30 more minutes?